Chapter 4: Queues

Technical / interview questions, exercises, multiple choice questions and answer key for Chapter 4 (Queues) of Data Structures.

Programming Exercises

1. Implement a queue using an array. The queue should support the following operations: • enqueue(item): Inserts an item into the queue. • dequeue(): Removes and returns the front item from the queue. • front(): Returns the front item of the queue without removing it. • isEmpty(): Checks whether the queue is empty.
2. Implement a queue using a linked list. The queue should support the same operations as the array-based queue.
3. Write a program that demonstrates the first-in, first-out (FIFO) property of queues. The program should enqueue a series of numbers, dequeue them, and print them to the console.
4. Write a program that demonstrates the breadth-first search (BFS) algorithm using a queue. The program should take a graph as input and find the shortest path from a starting node to a goal node.
5. Implement a simple scheduling algorithm using a queue. The program should simulate the execution of a set of tasks, each with a specified arrival time and execution time.
6. Implement a traffic light controller using a queue. The program should simulate the traffic flow at an intersection with multiple lanes.
7. Implement a call center simulation using a queue. The program should simulate the incoming calls to a call center and the distribution of calls to available agents.
8. Write a program that implements the hot potato problem using a queue. The hot potato problem is a classic game where players pass around a hot potato, and the player who has the potato when the timer runs out is eliminated.
9. Write a program that implements the producer-consumer problem using queues. The producer-consumer problem is a classic concurrency problem where multiple producers generate data and multiple consumers consume it.
10. Implement a circular queue. A circular queue uses a fixed-size array to store its elements, but it wraps around when the array is full.
11. Implement a priority queue. A priority queue stores elements with associated priorities, and elements are dequeued in order of decreasing priority.

Technical / Interview Questions

1. What is a queue, and how does it differ from other data structures like stacks?
2. Explain the concept of front and rear in a queue.
3. How is a queue different from a priority queue?
4. Discuss the key properties of a queue.
5. Explain the enqueue and dequeue operations in a queue.
6. How would you check if a queue is empty? Can the same method be used to check if it's full?
7. Describe the advantages and disadvantages of implementing a queue using an array.
8. What are the advantages of using a linked list to implement a queue?
9. Explain how you would handle overflow and underflow conditions in an array-based queue.
10. How is a queue used in the breadth-first search algorithm?

11. Can you explain why BFS is particularly suitable for finding the shortest path in an unweighted graph?
12. In the context of task scheduling, how can a priority queue be beneficial over a regular queue?
13. Discuss a scenario where a task with lower priority is dequeued before a higher-priority task. How would you address this?
14. How can queues be applied in managing traffic flow at intersections?
15. Explain how a queue-based system can help avoid traffic congestion.
16. In a call center scenario, how can queues be employed to manage incoming calls efficiently?
17. What considerations should be made when designing a queue system for handling diverse types of calls?

Multiple Choice Questions

1. What is the fundamental characteristic of a queue?

A. Last In, First Out (LIFO)
B. First In, First Out (FIFO)
C. Random access
D. Unordered storage

2. Why is a queue often compared to a line of people waiting?

A. Because it allows random access to elements
B. Because it follows a Last In, First Out (LIFO) order
C. Because it follows a First In, First Out (FIFO) order
D. Because it prioritizes elements based on size

3. What is the purpose of the enqueue operation in a queue?

A. To remove an element from the queue
B. To add an element to the front of the queue
C. To add an element to the rear of the queue
D. To check if the queue is full

4. In a queue, what is the role of the rear pointer?

A. Points to the first element
B. Points to the last element
C. Points to the middle element
D. Points to a random element

5. What is a potential disadvantage of using arrays to implement a queue?

A. Dynamic resizing
B. Limited size and potential overflow
C. Efficient random access
D. Automatic resizing

6. Why might linked lists be preferred for implementing queues?

A. Fixed size
B. Limited memory usage
C. Dynamic size and easy to expand
D. Efficient random access

7. How is a queue utilized in the breadth-first search (BFS) algorithm?

A. To store nodes at the deepest level
B. To randomly access nodes
C. To store nodes at each level
D. To store nodes in reverse order

8. In a traffic system, why is a first-come, first-served approach beneficial for managing vehicle flow?

A. It allows vehicles to pass randomly
B. It prioritizes vehicles from left to right
C. It ensures fair treatment based on arrival time
D. It prioritizes vehicles with even license plate numbers

9. In a task scheduling system, what is the purpose of prioritizing tasks?

A. To make the tasks look organized
B. To ensure higher-priority tasks are executed first
C. To minimize the number of tasks
D. To randomize the order of task execution

10. How does a priority queue differ from a regular queue in a scheduling system?

A. It allows tasks to be executed in any order
B. It executes tasks based on their size
C. It considers task priority during dequeue
D. It restricts the number of tasks in the queue

11. In a traffic system, why is a queue-based approach essential for managing vehicle intersections?

A. It allows vehicles to pass randomly
B. It prioritizes vehicles from left to right
C. It follows a first-come, first-served approach based on traffic rules
D. It prioritizes vehicles with odd license plate numbers

12. How can queues help avoid traffic congestion at intersections?

A. By allowing random access to vehicles
B. By prioritizing vehicles with higher speeds
C. By adhering to traffic rules and using a first-come, first-served approach
D. By skipping vehicles without proper documentation

13. In a call center queue system, what does the process of enqueueing represent?

A. Hanging up a call
B. Placing a call on hold
C. Ending a call
D. Receiving an incoming call

14. How can a call center use queues to manage different types of calls effectively?

A. By ignoring low-priority calls
B. By dequeuing calls randomly
C. By using multiple queues based on call types
D. By assigning the same priority to all calls

Answer Key

Q	Ans	Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	B	2	C	3	C	4	B	5	B
6	C	7	C	8	C	9	B	10	C

11	C	12	C	13	D	14	C		